

SMART DATA · DATABASE PRACTICE

학사관리 시스템 데이터베이스 종합실습

작성자

최재은

제출일

2026년 7월 22일

실습 환경

PostgreSQL 17.10 · DBeaver · psql

PostgreSQL 접속 확인부터 스키마·테이블 구성, ERD 설계, 제약조건 적용, 기초 조회, 함수 활용, JOIN까지의 종합 실습 결과를 정리하였다.

1. PostgreSQL 환경 구성 및 접속 확인

psql을 통해 현재 데이터베이스, 스키마, 사용자 및 PostgreSQL 버전을 조회하여 접속 환경을 확인하였다.

```
~ psql -P pager=off -d jaem -c "SELECT current_database(), current_schema(), current_user, version();"
current_database | current_schema | current_user | version
-----
jaem             | public        | jaem         | PostgreSQL 17.10 (Homebrew) on aarch64-apple-darwin25.4.0, compiled by Apple clang version 21.0.0 (clang-2100.0.123.102), 64-bit
(1개 행)
```

그림 1. PostgreSQL 접속 및 실행 환경 확인

1.1 스키마 및 테이블 구성 확인

jaem 데이터베이스의 univ 스키마에 학사관리용 4개 테이블을 구성하였다. 모든 테이블의 소유자는 jaem 이다.

```
~ psql -P pager=off -d jaem -c "\dt univ.*"
           릴레이션 목록
스키마 | 이름      | 형태 | 소유주
-----+-----+-----+-----
univ   | courses   | 테이블 | jaem
univ   | enrollments | 테이블 | jaem
univ   | majors    | 테이블 | jaem
univ   | students  | 테이블 | jaem
(4개 행)
```

```
~ psql -P pager=off -d jaem -c "SELECT 'courses', COUNT(*) FROM univ.courses"
table_name | row_count
-----
majors     | 4
students   | 10
courses    | 10
enrollments | 15
(4개 행)
```

그림 2. univ 스키마의 테이블 목록과 데이터 건수

데이터 입력 확인

students 10건, courses 10건, enrollments 15건으로 과제의 최소 10건 조건을 충족하였다. majors는 기존 정보 성격의 테이블로 4개 학과 데이터를 입력하였다.

2. 학사관리 시스템 ERD

학생과 과목 사이의 다대다(N:M) 관계를 수강신청(enrollments) 교차 테이블로 분리하여 각 관계를 1:N으로 표현하였다.

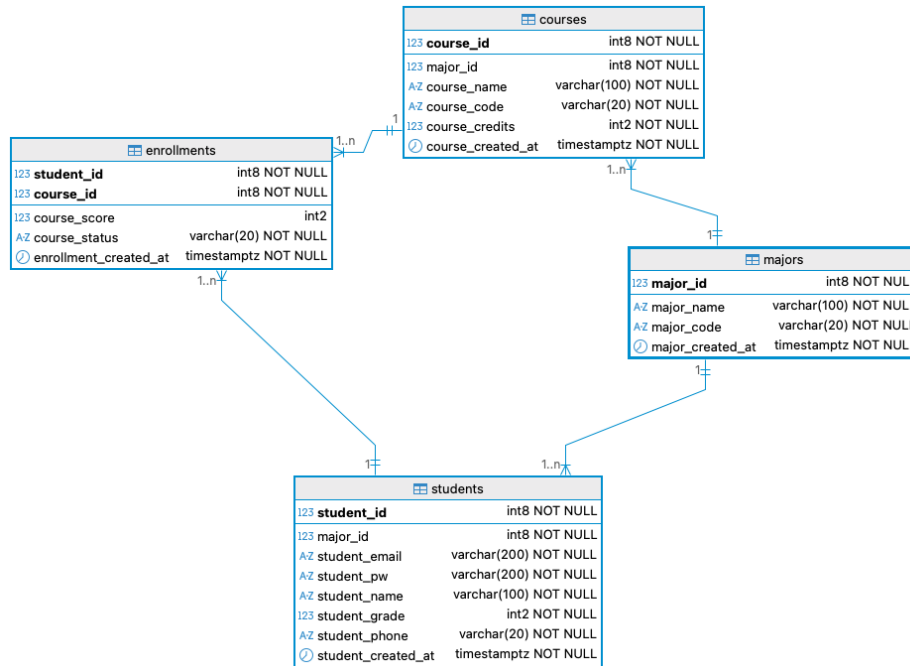


그림 3. 학사관리 시스템 ERD

ERD 범례

PK: 기본키(Primary Key) — 각 행을 고유하게 식별

FK: 외래키(Foreign Key) — 다른 테이블의 PK를 참조

1:N: 한 개의 부모 행에 여러 자식 행이 연결

enrollments: students와 courses의 N:M 관계를 해소하는 교차 테이블

관계: majors 1:N students / majors 1:N courses / students 1:N enrollments / courses 1:N enrollments

3. DDL 작성 및 제약조건 적용

PostgreSQL 문법에 맞게 IDENTITY 자동 증가, PK, FK, UNIQUE, NOT NULL, CHECK, DEFAULT 제약조건을 적용하였다.

```
CREATE TABLE majors (
    major_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    major_name VARCHAR(100) NOT NULL UNIQUE,
    major_code VARCHAR(20) NOT NULL UNIQUE,
    major_created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE students (
    student_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    major_id BIGINT NOT NULL REFERENCES majors(major_id),
    student_email VARCHAR(200) NOT NULL UNIQUE,
    student_pw VARCHAR(200) NOT NULL,
    student_name VARCHAR(100) NOT NULL,
    student_grade SMALLINT NOT NULL DEFAULT 1
    CHECK (student_grade BETWEEN 1 AND 4),
    student_created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE courses (
    course_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    major_id BIGINT NOT NULL REFERENCES majors(major_id),
    course_name VARCHAR(100) NOT NULL,
    course_code VARCHAR(20) NOT NULL UNIQUE,
    course_credits SMALLINT NOT NULL
    CHECK (course_credits BETWEEN 1 AND 6),
    course_created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

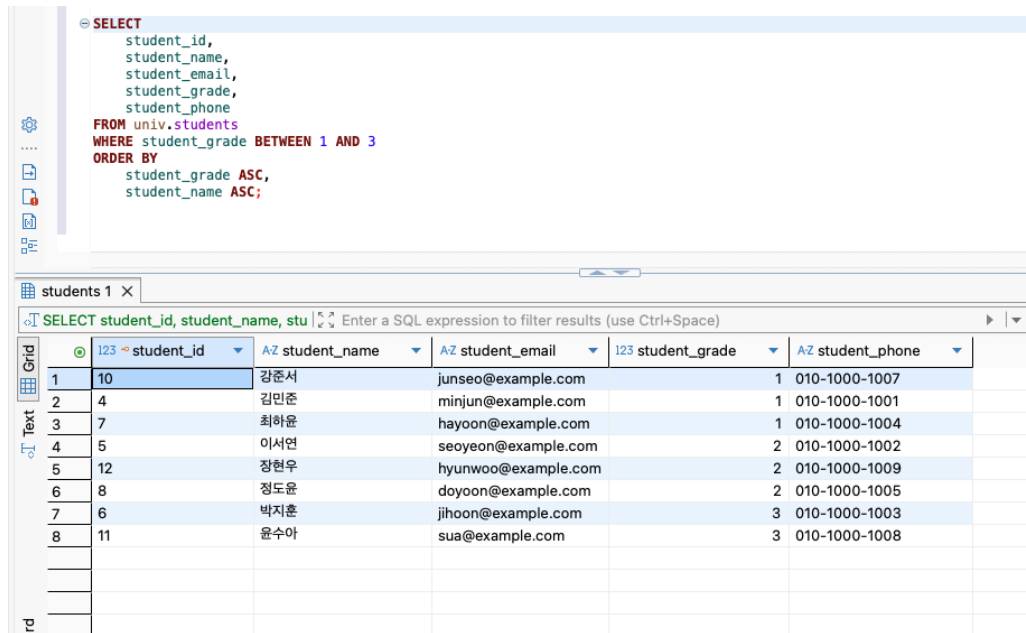
CREATE TABLE enrollments (
    student_id BIGINT NOT NULL
    REFERENCES students(student_id)
    ON DELETE CASCADE,
    course_id BIGINT NOT NULL
    REFERENCES courses(course_id)
    ON DELETE CASCADE,
    course_score SMALLINT
    CHECK (course_score BETWEEN 0 AND 100),
    course_status VARCHAR(20) NOT NULL
    DEFAULT 'ENROLLED'
    CHECK (
        course_status IN (
            'ENROLLED',
            'COMPLETED',
            'CANCELLED'
        )
    ),
    enrollment_created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (student_id, course_id)
);
```

그림 4. CREATE TABLE 및 제약조건 정의

테이블	주요 키	역할
majors	major_id	학과 기준 정보
students	student_id / major_id(FK)	학생 기본 정보
courses	course_id / major_id(FK)	과목 기본 정보
enrollments	student_id + course_id	수강 관계·점수·상태

4. SELECT · WHERE · ORDER BY 기초 조회

students 테이블에서 1~3학년 학생만 조회하고, 학년 오름차순 후 이름 오름차순으로 정렬하였다.



```
SELECT
  student_id,
  student_name,
  student_email,
  student_grade,
  student_phone
FROM univ.students
WHERE student_grade BETWEEN 1 AND 3
ORDER BY
  student_grade ASC,
  student_name ASC;
```

	student_id	student_name	student_email	student_grade	student_phone
1	10	강준서	junseo@example.com	1	010-1000-1007
2	4	김민준	minjun@example.com	1	010-1000-1001
3	7	최하윤	hayoon@example.com	1	010-1000-1004
4	5	이서연	seoyeon@example.com	2	010-1000-1002
5	12	장현우	hyunwoo@example.com	2	010-1000-1009
6	8	정도윤	doyoon@example.com	2	010-1000-1005
7	6	박지훈	jihoon@example.com	3	010-1000-1003
8	11	윤수아	sua@example.com	3	010-1000-1008

그림 5. 학생 조건 조회 및 정렬 결과

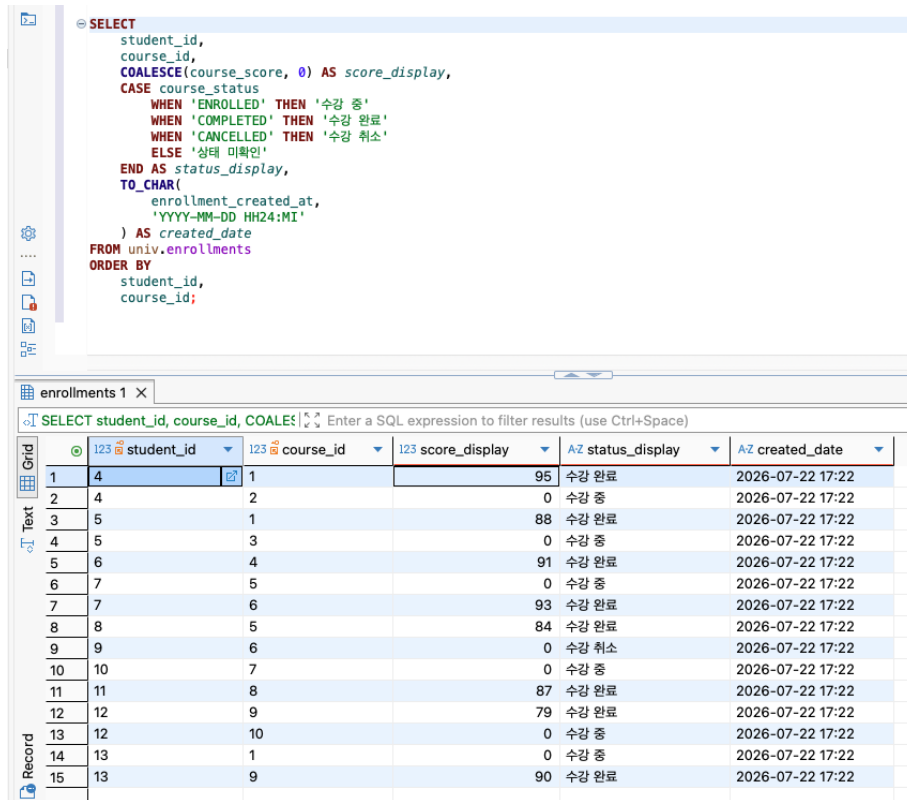
쿼리 해석

WHERE student_grade BETWEEN 1 AND 3: 1학년부터 3학년까지 필터링

ORDER BY student_grade ASC, student_name ASC: 학년별 정렬 후 같은 학년에는 이름순 정렬

5. COALESCE · CASE · 날짜 함수 활용

수강신청 데이터의 NULL 점수는 COALESCE로 0으로 표시하고, 영문 상태 코드는 CASE 표현식으로 한글화하였다. TO_CHAR로 생성 시각을 읽기 쉬운 형식으로 변환하였다.



```
SELECT
  student_id,
  course_id,
  COALESCE(course_score, 0) AS score_display,
  CASE course_status
    WHEN 'ENROLLED' THEN '수강 중'
    WHEN 'COMPLETED' THEN '수강 완료'
    WHEN 'CANCELLED' THEN '수강 취소'
    ELSE '상태 미확인'
  END AS status_display,
  TO_CHAR(
    enrollment_created_at,
    'YYYY-MM-DD HH24:MI'
  ) AS created_date
FROM univ.enrollments
ORDER BY
  student_id,
  course_id;
```

	student_id	course_id	score_display	status_display	created_date
1	4	1	95	수강 완료	2026-07-22 17:22
2	4	2	0	수강 중	2026-07-22 17:22
3	5	1	88	수강 완료	2026-07-22 17:22
4	5	3	0	수강 중	2026-07-22 17:22
5	6	4	91	수강 완료	2026-07-22 17:22
6	7	5	0	수강 중	2026-07-22 17:22
7	7	6	93	수강 완료	2026-07-22 17:22
8	8	5	84	수강 완료	2026-07-22 17:22
9	9	6	0	수강 취소	2026-07-22 17:22
10	10	7	0	수강 중	2026-07-22 17:22
11	11	8	87	수강 완료	2026-07-22 17:22
12	12	9	79	수강 완료	2026-07-22 17:22
13	12	10	0	수강 중	2026-07-22 17:22
14	13	1	0	수강 중	2026-07-22 17:22
15	13	9	90	수강 완료	2026-07-22 17:22

그림 6. NULL 처리, 상태 변환 및 날짜 포매팅 결과

함수별 역할

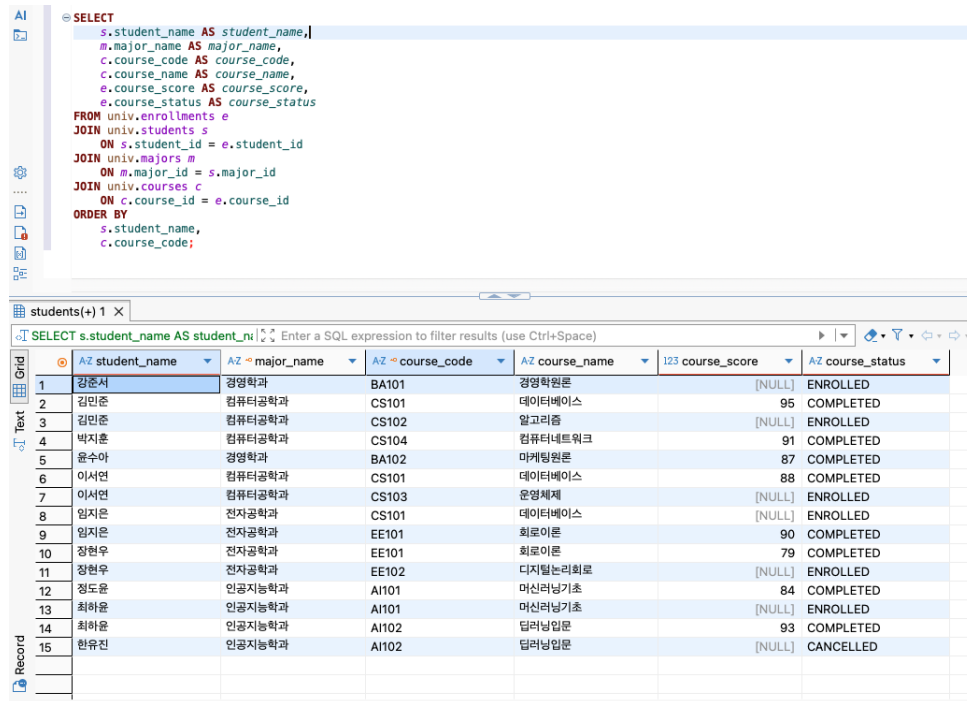
COALESCE(course_score, 0): 아직 점수가 없는 NULL 값을 0으로 표시

CASE course_status: ENROLLED / COMPLETED / CANCELLED를 수강 중 / 수강 완료 / 수강 취소로 변환

TO_CHAR(enrollment_created_at, 'YYYY-MM-DD HH24:MI'): 일시를 지정한 문자열 형식으로 출력

6. 수강신청 교차 테이블 JOIN 조회

enrollments를 중심으로 students, majors, courses를 INNER JOIN하여 학생명, 소속 학과, 과목 코드, 과목명, 점수와 수강 상태를 한 번에 조회하였다.



```
SELECT
  s.student_name AS student_name,
  m.major_name AS major_name,
  c.course_code AS course_code,
  c.course_name AS course_name,
  e.course_score AS course_score,
  e.course_status AS course_status
FROM univ.enrollments e
JOIN univ.students s
  ON s.student_id = e.student_id
JOIN univ.majors m
  ON m.major_id = s.major_id
JOIN univ.courses c
  ON c.course_id = e.course_id
ORDER BY
  s.student_name,
  c.course_code;
```

Grid	Text	Record	AZ student_name	AZ major_name	AZ course_code	AZ course_name	123 course_score	AZ course_status
1			강준서	경영학과	BA101	경영학원론	[NULL]	ENROLLED
2			김민준	컴퓨터공학과	CS101	데이터베이스	95	COMPLETED
3			김민준	컴퓨터공학과	CS102	알고리즘	[NULL]	ENROLLED
4			박지훈	컴퓨터공학과	CS104	컴퓨터네트워크	91	COMPLETED
5			윤수아	경영학과	BA102	마케팅원론	87	COMPLETED
6			이서연	컴퓨터공학과	CS101	데이터베이스	88	COMPLETED
7			이서연	컴퓨터공학과	CS103	운영체제	[NULL]	ENROLLED
8			임지은	전자공학과	CS101	데이터베이스	[NULL]	ENROLLED
9			임지은	전자공학과	EE101	회로이론	90	COMPLETED
10			장현우	전자공학과	EE101	회로이론	79	COMPLETED
11			장현우	전자공학과	EE102	디지털논리회로	[NULL]	ENROLLED
12			정도윤	인공지능학과	AI101	머신러닝기초	84	COMPLETED
13			최하윤	인공지능학과	AI101	머신러닝기초	[NULL]	ENROLLED
14			최하윤	인공지능학과	AI102	딥러닝입문	93	COMPLETED
15			한유진	인공지능학과	AI102	딥러닝입문	[NULL]	CANCELLED

그림 7. 네 개 테이블의 JOIN 결과

JOIN 구조

enrollments.student_id = students.student_id

students.major_id = majors.major_id

enrollments.course_id = courses.course_id

7. 실습 결과 및 정리

ERD에서 정의한 관계를 실제 PostgreSQL 스키마로 구현하고, 샘플 데이터 입력과 조회를 통해 PK·FK 관계가 정상적으로 동작함을 확인하였다. 특히 enrollments 교차 테이블을 이용해 학생과 과목의 N:M 관계를 표현했으며, 조건 조회, NULL 처리, 상태 코드 변환, 날짜 포매팅 및 다중 테이블 JOIN을 수행하였다.